Object Oriented Programming I

September 18, 2024

Content

Classes, Objects, Member Functions and Data Members

Defining a Class with a Member Function

Defining a Member Function with a Parameter

Data Members, set Functions and get Functions

Initializing Objects with Constructors

Placing a Class in a Separate File for Reusability

Separating Interface from Implementation

Validating Data with set Functions

The Automobile as an Object

- To help you understand objects and their contents, let's begin with a simple analogy. Suppose you want to drive a car and make it go faster by pressing its accelerator pedal.
- What must happen before you can do this?
- Well, before you can drive a car, some one has to design it.
- A car typically begins as engineering drawings,similar to the blueprints that describe the design of a house.
- These drawings include the design for an accelerator pedal.The pedal hides from the driver the complex mechanisms that actually make the car go faster, just as the brake pedal hides the mechanisms that slow the car,and the steering wheel "hides" the mechanisms that turn the car.

The Automobile as an Object

- This enables people with little or no knowledge of how engines,braking and steering mechanisms work to drive a car easily, simply by using the accelerator pedal, the break pedal, the steering wheel, the transmission mechanism and other such simple and user-friendly "interfaces" to the car's complex internal mechanism.
- You cannot drive a car's engineering drawings.
- Before you can drive a car, it must be built from the engineering drawings that describe it.
- A completed car has an actual accelerator pedal to make the car go faster.
- But even that's not enough. The car won't accelerate on its own,so the driver must press the pedal to accelerate the car to tell the car to go faster.

Member Functions and Classes

- Let's use our car example to introduce some key object-oriented programming concepts.
- Performing a task in a program requires a member function,which houses the program statements that actually perform its task.
- The member function hides these statements from its user, just as the accelerator pedal of a car hides from the driver the mechanisms of making the car go faster.
- In C++, we create a program unit called a class to house the set of member functions that perform the class's tasks.
- A class is similar in concept to a car's engineering drawings,which house the design of an accelerator pedal, steering wheel,and so on.

Instantiation

- Just as some one has to build a car from its engineering drawings before you can actually drive a car, you must build an object of a class before a program can perform the tasks that the class's member functions define.
- The process of doing this is called instantiation.
- An object is then referred to as an instance of its class.
- Note also that just as many cars can be built from the same engineering drawing, many objects can be built from the same class.

Messages and Member Function Calls

- When you drive a car, pressing its gas pedal sends a message to the car to perform a task that is to go faster.
- Similarly,you send messages to an object. Each message is implemented as a member function call that tells a member function of the object to perform its task.

Attributes and Data Members

- A car,besides having capabilities to accomplish tasks,also has attributes,such as its color, its number of doors,the amount of gas in its tank, its current speed and its record of total miles driven.
- Like its capabilities, the car's attributes are represented as part of its design in its engineering diagrams.
- As you drive an actual car, these attributes are carried along with the car. Every car maintains its own attributes. For example, each car knows how much gas is in its own gas tank,but not how much is in the tanks of other cars.
- An object,similarly,has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class.
- Attributes are specified by the class's data members.

Defining a Class with a Member Function

```cpp
/* ******************************************************
 * Filename     :   defineClass.cpp
 * Author       :   Burak Kaleci
 * Date         :   26.09.2018
 * Description  :   An example program to define a class
 * ******************************************************/

// includes input/output stream header
// To output data to the screen
#include <iostream>

using namespace std;

class GradeBook
{
public:
    //  function that displays a welcome message to the GradeBook user
    void displayMessage()
    {
        cout << "Welcome to the Grade Book!" << endl;
    }
};

// function main begins program execution each program must include main function
main()
{
    // create aGradeBook object named myGradeBook
    GradeBook myGradeBook;
    // call object's displayMessage function
    myGradeBook.displayMessage();
}
```

We begin with an example that consists of **class GradeBook** and a **main function** that creates a GradeBook object. Function main uses this object and its member function to display a message on the screen welcoming the instructor to the grade-book program.

Class **GradeBook**

- Before function main can create a GradeBook object,we must tell the compiler what member functions and data members belong to the class.
- The GradeBook class definition (lines 14-22) contains a member function called displayMessage (lines 18-21) that displays a message on the screen (line 20).
- We need to make an object of class GradeBook (line 28) and call its displayMessage member function (line 29) to get line 20 to execute and display the welcome message.

Class **GradeBook**

- The class definition begins in line 14 with the **keyword class** followed by the class name GradeBook.
- By convention, the name of a user-defined class begins with a capital letter, and for readability, each subsequent word in the class name begins with a capital letter.
- Every class's body is enclosed in a pair of left and right braces ({ and }), as in lines 15 and 22. The class definition terminates with a semicolon (line 22).

Class **GradeBook**

- Line 16 contains the **keyword public**,which is an access specifier.
- Lines 18-21 define member function displayMessage.
- This member function appears after access specifier **public:** to indicate that the function is "available to the public" which means that,it can be called by other functions in the program (such as main), and by member functions of other classes(if there are any).
- Access specifiers are always followed by a colon (:).

Class **GradeBook**

- The name of the member function is displayMessage
- By convention,function names begin with a lowercase first letter and all subsequent words in the name begin with a capital letter.
- The body of a function contains statements that perform the function's task.In this case, member function displayMessage contains one statement (line 20) that displays the message "Welcome to theGrade Book!".
- After this statement executes, the function has completed its task.

Testing Class **GradeBook**

- In this program, we'd like to call class GradeBook's displayMessage member function to display the welcome message.
- **Typically,you cannot call a member function of a class until you create an object of that class.**
- Line 28 creates an object of class GradeBook called myGradeBook.
- The variable's type is GradeBook which is the class we defined in lines 14-22.
- When we declare variables of type int the compiler knows what int is. It's a fundamental type that's "built into" C++.

Testing Class **GradeBook**

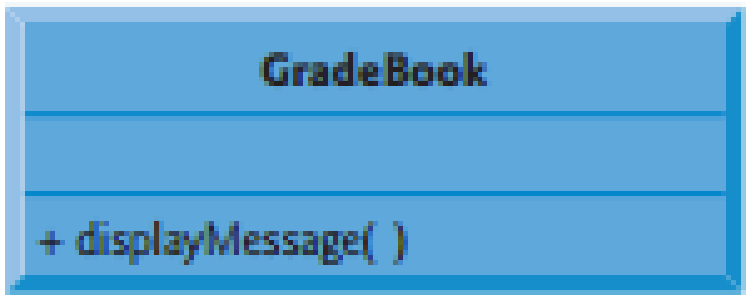
- In line 28, however, the compiler does not automatically know what type GradeBook is. It's a user-defined type. We tell the compiler what GradeBook is by including the class definition (lines 14-22).
- If we omitted these lines, the compiler would issue an error message. Each class you create becomes a new type that can be used to create objects.
- Line 29 calls the member function displayMessage using variable myGradeBook followed by the dot operator (.).
- This call causes the displayMessage function to perform its task.
- At the beginning of line 29, "myGradeBook."indicates that main should use the GradeBook object that was created in line 28.

UML Class Diagram for Class **GradeBook**

- The UML is a standardized graphical language used by software developers to represent their object-oriented systems.
- In the UML, each class is modeled in a UML class diagram as a rectangle with three compartments.
- The top compartment contains the class's name centered horizontally and in bold face type.
- The middle compartment contains the class's attributes,which correspond to data members in C++.
- The bottom compartment contains the class's operations, which correspond to member functions in C++.
- The UML models operations by listing the operation name followed by a set of parentheses.

UML Class Diagram for Class **GradeBook**

- The middle compartment is currently empty, because class GradeBook does not have any attributes.
- Class GradeBook has only one member function, displayMessage,so the bottom compartment lists one operation with this name.
- The plus sign(+) in front of the operation name indicates that displayMessage is a public operation.



Defining a Member Function with a Parameter

- A member function can require one or more parameters that represent additional data it needs to perform its task.
- A function call supplies values which are called arguments, for each of the function's parameters.
- Our next example redefines class GradeBook with a displayMessage member function that displays the course name as part of the welcome message.
- The new version of displayMessage requires a parameter(courseName) that represents the course name to output.

Defining a Member Function with a Parameter

```cpp
/* ******************************************************
 * Filename     :   parameter.cpp
 * Author       :   Burak Kaleci
 * Date         :   26.09.2018
 * Description  :   An example program to define a member function with a parameter
 * ******************************************************/

// includes input/output stream header
// To output data to the screen
#include <iostream>
#include <string> // program uses C++ standard string class
using namespace std;

class GradeBook
{
public:
    //  function that displays a welcome message to the GradeBook user
    void displayMessage(string courseName)
    {
        cout << "Welcome to the Grade Book for " << courseName << "!" << endl;
    }
};

// function main begins program execution each program must include main function
main()
{
    // string of characters to store the course name
    string nameOfCourse;
    // create aGradeBook object named myGradeBook
    GradeBook myGradeBook;

    cout << "Please enter the course name:" << endl;
    cin >> nameOfCourse;

    // call object's displayMessage function and pass nameOfCourse as an argument
    myGradeBook.displayMessage(nameOfCourse);
}
```

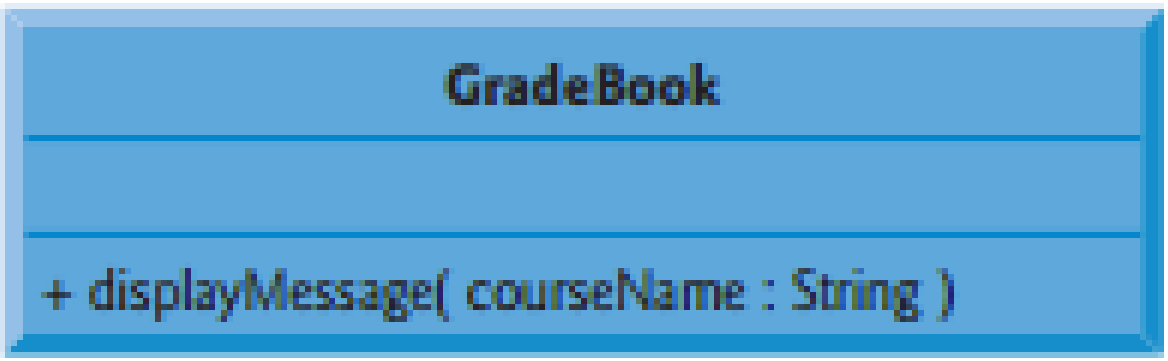
String

- Line 28 creates a variable of type string called nameOfCourse that will be used to store the course name entered by the user.
- A variable of type string represents a string of characters such as "AdvancedProgramming".
- A string is actually an **object** of the C++ Standard Library class string.
- This class is defined in header <string>,and the name string,like cout,belongs to namespace std.
- To enable lines 18 and 28 to compile,line 11 includes the <string> header.The using directive in line 12 allows us to simply write string in line 28 rather than std::string.

Defining a Member Function with a Parameter

- Line 30 creates an object of class GradeBook named myGradeBook.
- Line 32 prompts the user to enter a course name.
- Line 33 reads the name from the user and assigns it to the nameOfCourse variable.
- Line 36 calls myGradeBook's displayMessage member function. The nameOfCourse variable in parentheses is the argument that's passed to member function displayMessage so that it can perform its task.
- The value of variable nameOfCourse in main is copied to member function displayMessage's parameter courseName in line 18.
- When you execute this program, member function displayMessage outputs as part of the welcome message the course name you type.

Updated UML Class Diagram for Class GradeBook



Local Variables

- Variables declared in a function definition's body are known as **local variables** and **can be used only from the line of their declaration in the function to closing right brace (}) of the block in which they're declared.**
- A local variable must be declared before it can be used in a function.
- A local variable cannot be accessed outside the function in which it's declared.
- When a function terminates,the values of its local variables are lost.

Data Members

- A class normally consists of one or more member functions that manipulate the attributes that belong to a particular object of the class.
- **Attributes are represented as variables in a class definition.**
- **Such variables are called data members** and are declared inside a class definition but outside the bodies of the class's member-function definitions.
- Each object of a class maintains **its own copy of its attributes** in memory.
- These attributes exist throughout the life of the object.

GradeBook Class with a Data Member,a set Function and a get Function

```
14  class GradeBook
15  {
16  public:
17
18      // function that sets the course name
19      void setCourseName(string name)
20      {
21          courseName=name; // store the course name in the object
22      }
23
24      // function that gets the course name
25      string getCourseName()
26      {
27          return courseName; // return the object's courseName
28      }
29
30      //  function that displays a welcome message
31      void displayMessage()
32      {
33          cout << "Welcome to the Grade Book for " << getCourseName() << "!" << end
34      }
35
36  private:
37      string courseName; // coursenamefor this GradeBook
38  };
```

```
40  // function main begins program execution each program must include main functi
41  main()
42  {
43      // string of characters to store the course name
44      string nameOfCourse;
45      // create aGradeBook object named myGradeBook
46      GradeBook myGradeBook;
47
48      // display initial value of courseName
49      cout << "Initial course name is: "  << myGradeBook.getCourseName() << endl;
50
51      cout << "Please enter the course name:" << endl;
52      cin >> nameOfCourse;
53
54      // setthe course name
55      myGradeBook.setCourseName(nameOfCourse);
56
57      cout << endl;
58      // call object's displayMessage function
59      myGradeBook.displayMessage();
60  }
```

GradeBook Class with a Data Member,a set Function and a get Function

- In our next example, class GradeBook maintains the course name as a data member so that it can be used or modified at any time during a program's execution.
- The class contains member functions setCourseName, getCourseName and displayMessage.
- Member function setCourseName stores a course name in a GradeBook data member.
- Member function getCourseName obtains the course name from that data member.
- Member function displayMessage which now specifies no parameters still displays a welcome message that includes the course name. However,the function now obtains the course name by calling another function in the same class getCourseName.

GradeBook Class with a Data Member,a set Function and a get Function

- A typical instructor teaches multiple courses,each with its own course name.
- Every instance(i.e.,object) of class GradeBook contains one copy of each of the class's data members if there are two GradeBook objects,each has its own copy of courseName (one per object).
- A benefit of making courseName a data member is that all the member functions of the class can manipulate any data members that appear in the class definition.

Access Specifiers public and private

- Most data-member declarations appear after the private access specifier.
- Variables or functions declared after access specifier private are accessible only to member functions of the class for which they're declared.
- Thus, data member courseName can be used only in member functions setCourseName, getCourseName and displayMessage of class GradeBook.
- Declaring data members with access specifier private is known as data hiding. When a program creates a GradeBook object,data member courseName is encapsulated (hidden) in the object and can be accessed only by member functions of the object's class.

Member Functions setCourseName and getCourseName

- In class GradeBook,member functions setCourseName and getCourseName manipulate the data member courseName directly.
- Member function setCourseName (lines 19-22) does not return any data when it completes its task, so its return type is void.The member function receives one parameter, name, which represents the course name that will be passed to it as an argument. Line 21 assigns name to data member courseName.
- Member function getCourseName (lines 25-28) returns a particular GradeBook object's courseName. The member function has an empty parameter list, so it does not require additional data to perform its task. The function specifies that it returns a string. When a function that specifies a return type other than void is called and completes its task,the function uses a return statement (as in line 27) to return a result to its calling function.

Testing Class GradeBook

- The main function creates one object of class GradeBook and uses each of its member functions.
- Line 46 creates a GradeBook object named myGradeBook. Line 49 display the initial course name by calling the object's getCourseName member function. The first line of the output does not show a course name, because the object's courseName data member is initially empty, by default.
- Line 55 calls object myGradeBook's setCourseName member function and supplies nameOfCourse as the function's argument.
- When the function is called, the argument's value is copied to parameter name (line 19) of member function setCourseName.Then the parameter's value is assigned to data member courseName (line 21).
- Line 53 calls object myGradeBook's displayMessage member function to display the welcome message containing the course name.

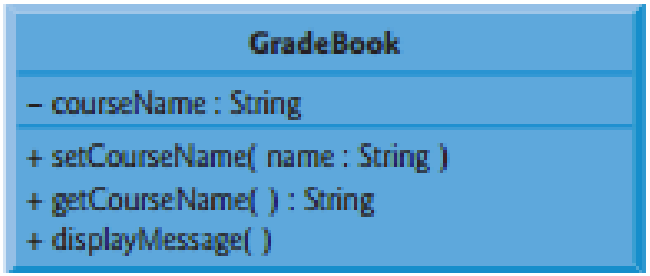
Software Engineering with Set and Get Functions

- A class's private data members can be manipulated only by member functions of that class.
- So a client of an object calls the class's public member functions to request the class's services for particular objects of the class.
- This is why the statements in function main call member functions setCourseName, getCourseName and displayMessage on a GradeBook object.
- Classes often provide public member functions to allow clients of the class to set (i.e.,assign values to) or get (i.e.,obtain the values of) private data members.
- Recall that declaring data members with access specifier private enforces data hiding. Providing public set and get functions allows clients of a class to access the hidden data, but only indirectly.
- The client knows that it's attempting to modify or obtain an object's data,but the client does not know how the object performs these operations.

Software Engineering with Set and Get Functions

- The set and get functions of a class also should be used by other member functions within the class to manipulate the class's private data,although these member functions can access the private data directly.
- Member function displayMessage calls member function getCourseName to obtain the value of data member courseName for display purposes, even though displayMessage can access courseName directly, accessing a data member via its get function creates a better, more robust class.
- If we decide to change the data member courseName in some way, the displayMessage definition will not require modification; only the bodies of the get and set functions that directly manipulate the data member will need to change.
- For example, suppose we want to represent the course name as two separatedata members such as courseNumber (e.g., "CS101")and courseTitle (e.g., "AdvancedProgramming"). Member function displayMessage can still issue a single call to member function getCourseName to obtain the full course name to display as part of the welcome message.
- In this case, getCourseName would need to build and return a string containing the courseNumber followed by the courseTitle.

Updated UML Class Diagram for Class GradeBook



- The UML represents data members as attributes by listing the attribute name, followed by a colon and the attribute type.
- Data member courseName is private in C++, so the class diagram lists a minus sign (–) in front of the corresponding attribute's name.
- The minus sign in the UML is equivalent to the private access specifier in C++.
- The UML indicates the return type of an operation by placing a colon and the return type after the parentheses following the operation name.

Constructors

- Each class you declare can provide a constructor that can be used to initialize an object of the class when the object is created.
- A constructor is a special member function that must be defined with the same name as the class, so that the compiler can distinguish it from the class's other member functions.
- An important difference between constructors and other functions is that constructors cannot return values, so they cannot specify a return type (not even void).
- C++ requires a constructor call for each object that's created, which helps ensure that each object is initialized properly before it's used in a program.
- The constructor call occurs implicitly when the object is created. If a class does not explicitly include a constructor, the compiler provides a default constructor, that is, a constructor with no parameters.
- The default constructor provided by the compiler creates a GradeBook object without giving any initial values to the object's fundamental type data members.
- This is why the string data member courseName was initialized to the empty string. The default constructor for class string sets the string's value to the empty string.

Constructors

```
14  class GradeBook
15  {
16  public:
17
18      // constructor initializes courseName with string supplied
19      GradeBook(string name)
20      {
21          setCourseName(name); // call set function to initializ
22      }
23
24      // function that sets the course name
25      void setCourseName(string name)
26      {
27          courseName=name; // store the course name in the object
28      }
29
30      // function that gets the course name
31      string getCourseName()
32      {
33          return courseName; // return the object's courseName
34      }
35
36      //  function that displays a welcome message
37      void displayMessage()
38      {
39          cout << "Welcome to the Grade Book for " << getCourseName() << "!" << endl;
40      }
41
42  private:
```

```
47  main()
48  {
49
50      // create two GradeBook objects
51      GradeBook gradeBook1("EEE102_ComputerProgramming");
52      GradeBook gradeBook2("EEE103_AdvancedProgramming");
53
54      // display initial value of courseName for each GradeBook
55      cout << "gradeBook1 created for course: "  << gradeBook1.getCourseName()
56          << "\ngradeBook2 created for course: " << gradeBook2.getCourseName()
57          << endl;
58
59  }
```

Defining a Constructors

- Lines 19-22 define a constructor for class GradeBook. Notice that the constructor has the same name as its class, GradeBook.
- A constructor specifies in its parameter list the data it requires to perform its task. When you create a new object, you place this data in the parentheses that follow the object name (as we did in lines 51-52).
- Line 19 indicates that class GradeBook's constructor has a string parameter called name. Line 19 does not specify a return type, because constructors cannot return values (or even void).
- Line 21 in the constructor's body passes the constructor's parameter name to member function setCourseName (lines 25-28),which simply assigns the value of its parameter to data member courseName.

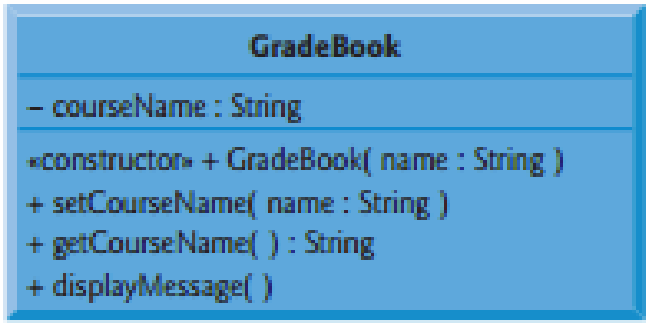
Defining a Constructors

- You might be wondering why we make the call to setCourseName in line 21 rather than the constructor certainly could perform the assignment courseName=name.
- We can modify setCourseName to perform validation (ensuring that, in this case,the courseName is 25 or fewer characters in length).
- Both the constructor (line 19) and the setCourseName function (line 25) use a parameter called name.You can use the same parameter names in different functions because the parameters are local to each function;they do not interfere with one another.

Testing Class GradeBook

- Line 51 creates and initializes GradeBook object gradeBook1. When this line executes, the GradeBook constructor (lines 19-22) is called (implicitly by C++) with the argument EEE102_ComputerProgramming to initialize gradeBook1's coursename.
- Line 52 repeats this process for GradeBook object gradeBook2,this time passing the argument EEE103_AdvancedProgramming to initialize gradeBook2's coursename.
- Lines 55-56 use each object's getCourseName member function to obtain the course names and show that they were indeed initialized when the objects were created. The output confirms that **each GradeBook object maintains its own copy of data member courseName**.

Updated UML Class Diagram for Class GradeBook



- Like operations,the UML models constructors in the third compartment of a class in a class diagram.
- To distinguish a constructor from a class's operations,the UML places the word "constructor" between guillemets before the constructor's name.
- By convention,you list the class's constructor before other operations in the third compartment.

Headers

- Programmers who wish to use our GradeBook class cannot simply include the file in another program.
- We know that the function main begins the execution of every program, and **every program must have exactly one main function**.
- So, placing main in the same file with a class definition prevents that class from being reused by other programs.
- Each of the previous examples consists of a single .cpp file, also known as a source-code file, that contains a GradeBook class definition and a main function.
- When building an object-oriented C++ program, it's customary to define reusable source code (such as a class) in a file that by convention has a .h filename extension which is known as a header.
- Programs use #include preprocessor directives to include headers and take advantage of reusable software components.
- Our next example separates the code into two files header and driver program.

Header

```
10  #include <iostream>
11  #include <string> // program uses C++ standard string class
12  using namespace std;
13
14  class GradeBook
15  {
16  public:
17
18      // constructor initializes courseName with string supplied as argument
19      GradeBook(string name)
20      {
21          setCourseName(name); // call set function to initialize courseName
22      }
23
24      // function that sets the course name
25      void setCourseName(string name)
26      {
27          courseName=name; // store the course name in the object
28      }
29
30      // function that gets the course name
31      string getCourseName()
32      {
33          return courseName; // return the object's courseName
34      }
35
36      //  function that displays a welcome message
37      void displayMessage()
38      {
39          cout << "Welcome to the Grade Book for " << getCourseName() << "!" << endl;
40      }
41
42  private:
43      string courseName; // coursenamefor this GradeBook
44  };
```

Notice that the header contains only the GradeBook class definition, the appropriate headers and a using directive.

Driver Program

```
 8  // includes input/output stream header
 9  // To output data to the screen
10  #include <iostream>
11  #include "constructors.h" // include definition of class GradeBoo
12  using namespace std;
13
14  // function main begins program execution each program must include main function
15  main()
16  {
17
18      // create two GradeBook objects
19      GradeBook gradeBook1("EEE102_ComputerProgramming");
20      GradeBook gradeBook2("EEE103_AdvancedProgramming");
21
22      // display initial value of courseName for each GradeBook
23      cout << "gradeBook1 created for course: "  << gradeBook1.getCourseName()
24          << "\ngradeBook2 created for course: " << gradeBook2.getCourseName()
25          << endl;
26
27  }
28
```

The main function that uses class GradeBook. To help you prepare for the larger programs, we often use a separate source-code file containing function main to test our classes (this is called a driver program).

Including a Header That Contains a User-Defined Class

- A header such as constructors.h cannot be used as a complete program, because it does not contain a main function.If you try to compile and link constructors.h by itself to create an executable application you can get an error message.
- This error indicates that the linker could not locate the program's main function. To test class GradeBook, you must write a separate source-code file containing a main function that instantiates and uses objects of the class.
- The compiler doesn't know what a GradeBook is because it's a user-defined type. In fact,the compiler doesn't even know the classes in the C++ Standard Library.
- To help it understand how to use a class, we must explicitly provide the compiler with the class's definition.
- This enables the compiler to determine the amount of memory that it must reserve for each object and ensure that a program calls a class's member functions correctly.

Including a Header That Contains a User-Defined Class

- To create GradeBook objects gradeBook1 and gradeBook2, the compiler must know the size of a GradeBook object.
- While objects conceptually contain data members and member functions, C++ objects actually contain only data. The compiler creates only one copy of the class's member functions and shares that copy among all the class's objects.
- Each object,of course, needs its own copy of the class's data members, because their contents can vary among objects.
- The member-function code,however,is not modifiable, so it can be shared among all objects of the class. Therefore, the size of an object depends on the amount of memory required to store the class's data members.
- By including constructors.h, we give the compiler access to the information it needs to determine the size of a GradeBook object and to determine whether objects of the class are used correctly.

Including a Header That Contains a User-Defined Class

- #include "constructors.h" instructs the C++ preprocessor to replace the directive with a copy of the contents of constructors.h before the program is compiled.
- When the driver program is compiled, it now contains the GradeBook class definition (because of the #include), and the compiler is able to determine how to create GradeBook objects and see that their member functions are called correctly.
- Now that the class definition is in a header(without a main function), we can include that header in any program that needs to reuse our GradeBook class.

How Headers Are Located

- Notice that the name of the constructors.h header in is enclosed in quotes("") rather than angle brackets (<>).
- Normally, a program's source-code files and user defined headers are placed in the same directory.
- When the preprocessor encounters a header name in quotes,it attempts to locate the header in the same directory as the file in which the #include directive appears.If the preprocessor cannot find the header in that directory, it searches for it in the same location(s) as the C++ Standard Library headers.
- When the preprocessor encounters a header name in angle brackets (e.g., <iostream>), it assumes that the header is part of the C++ Standard Library and does not look in the directory of the program that's being preprocessed.

Motivation

- Now that class GradeBook is defined in a header,the class is reusable.Unfortunately, placing a class definition in a header still reveals the entire implementation of the class to the class's clients.
- constructors.h is simply a text file that anyone can open and read. Conventional software engineering wisdom says that to use an object of a class, the client code needs to know only what member functions to call,what arguments to provide to each member function and what return type to expect from each member function.
- The client code does not need to know how those functions are implemented.
- If client code does know how a class is implemented, the programmer might write client code based on the class's implementation details.
- Ideally, if that implementation changes,the class's clients should not have to change. Hiding the class's implementation details makes it easier to change the class's implementation while minimizing,and hopefully eliminating, changes to client code.

Interface of a Class

- The interface of a class describes what services a class's clients can use and how to request those services, but not how the class carries out the services.
- A class's public interface consists of the class's public member functions.
- One can specify a class's interface by writing a class definition that lists only the member-function names, return types and parameter types.

Defining a Class's Interface with Function Prototypes

```
1  /* ***************************************************
2   * Filename     :   GradeBook.h
3   * Author       :   Burak Kaleci
4   * Date         :   27.09.2018
5   * Description  :   An example program
6   * ***************************************************/
7
8  // program uses C++ standard string class
9  #include <string>
10 using namespace std;
11
12 // GradeBook class definition
13 class GradeBook
14 {
15 public:
16
17     // constructor that initializes courseName
18     GradeBook(string);
19     // function that sets the course name
20     void setCourseName(string);
21     // function that gets the course name
22     string getCourseName();
23     //  function that displays a welcome message
24     void displayMessage();
25
26 private:
27     // course name for this GradeBook
28     string courseName;
29 };
30
```

- GradeBook.h now only consists of function prototypes that describe the class's public interface without revealing the class's member-function implementations.
- A function prototype is a declaration of a function that tells the compiler the function's name,its return type and the types of its parameters.Also, the header still specifies the class's private data member as well.
- Again, the compiler must know the data members of the class to determine how much memory to reserve for each object of the class.
- Including the header GradeBook.h in the driver code provides the compiler with the information it needs to ensure that the client code calls the member functions of class GradeBook correctly.

Defining Member Functions in a Separate Source-Code File

```cpp
/* ******************************************************
 * Filename     :   GradeBook.cpp
 * Author       :   Burak Kaleci
 * Date         :   27.09.2018
 * Description  :   An example program
 * ******************************************************/

// includes input/output stream header
// To output data to the screen
#include <iostream>
#include "GradeBook.h" // include definition of class GradeBook
using namespace std;


// constructor initializes courseName with string supplied as argument
GradeBook::GradeBook(string name)
{
    setCourseName(name); // call set function to initialize courseName
}

// function that sets the course name
void GradeBook::setCourseName(string name)
{
    courseName=name; // store the course name in the object
}

// function that gets the course name
string GradeBook::getCourseName()
{
    return courseName; // return the object's courseName
}

//  function that displays a welcome message
void GradeBook::displayMessage()
{
    cout << "Welcome to the Grade Book for " << getCourseName() << "!" << endl;
}

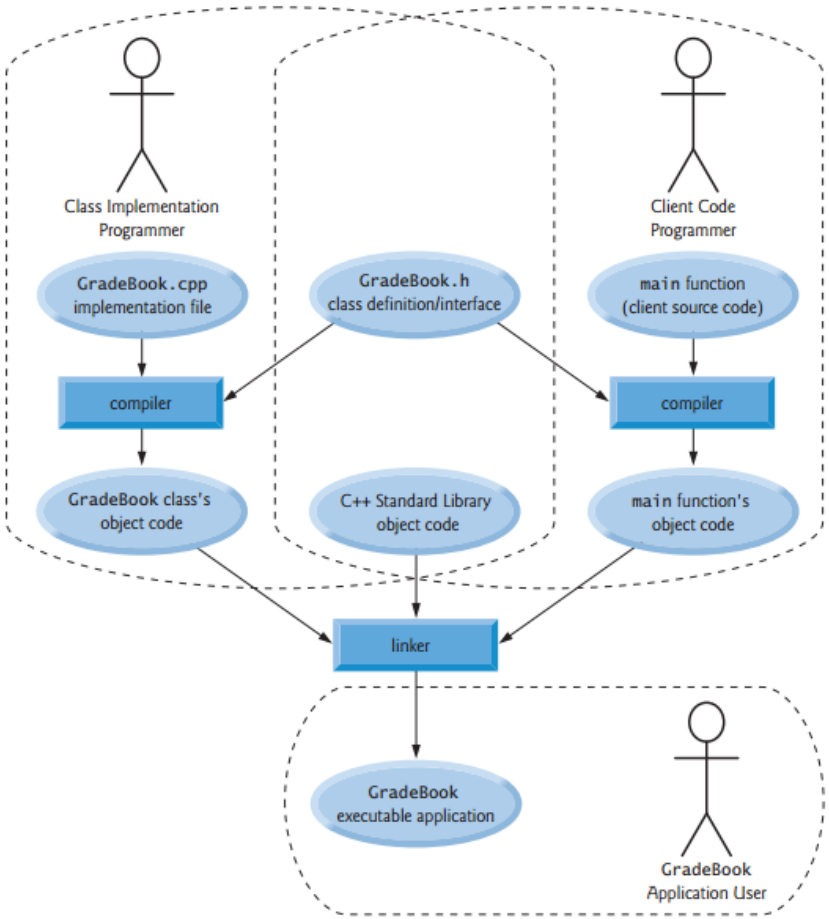
```

- Each member-function name in the function headers is preceded by the class name and ::, which is known as the binary scope resolution operator.
- This "ties" each member function to the(now separate) GradeBook class definition, which declares the class's member functions and data members.
- Without "GradeBook::" preceding each function name,these functions would not be recognized by the compiler as member functions of class GradeBook.

Defining Member Functions in a Separate Source-Code File

- The compiler would consider them "free" or "loose" functions, like main. These are also called global functions.
- Such functions cannot access GradeBook's private data or call the class's member functions,without specifying an object. So,the compiler would not be able to compile these functions.
- To indicate that the member functions in GradeBook.cpp are part of class GradeBook, we must first include the GradeBook.h header.
- This allows us to access the class name GradeBook in the GradeBook.cpp file.
- When compiling GradeBook.cpp, the compiler uses the information in GradeBook.h to ensure that
  - The first line of each member function matches its prototype in the GradeBook.h file.
  - Each member function knows about the class's data members and other member functions

The Compilation and Linking Process



Validating Data with set Functions

- In previous examples, class GradeBook defines member function setCourseName to simply assign a value received in its parameter name to data member courseName.
- This member function does not ensure that the course name adheres to any particular format or follows any other rules regarding what a "valid" course name looks like.
- Suppose that a university can print student transcripts containing course names of only 30 characters or less.
- If the university uses a system containing GradeBook objects to generate the transcripts, we might want class GradeBook to ensure that its data member courseName never contains more than 30 characters.

Validating the Course Name with setCourseName

```
21  // function that sets the course name
22  void GradeBook::setCourseName(string name)
23  {
24      if (name.length() <= 30) // if name has 30 or fewer characters
25          courseName=name; // store the course name in the object
26      else {
27          // set courseName to first 30 characters of parameter name
28          courseName =name.substr( 0, 30 ); // start at 0,length of 30
29          cout << "Name \"" << name<< "\" exceeds maximum length (30).\n"
30  << "Limiting courseName to first 30 characters.\n" << endl;
31      }
32  }
```

- The if statement in lines 24-25 determines whether parameter name contains a valid coursename(i.e., a string of 30 or fewer characters).
- If the course name is valid, line 25 stores it in data member courseName.
- Note the expression name.length() in line 24. The C++ Standard Library's string class defines a member function length that returns the number of characters in a string object.Parameter name is a string object, so the call name.length() returns the number of characters in name.

Validating the Course Name with setCourseName

- The else statement in lines 26-31 handles the case in which setCourseName receives an invalid coursename(i.e., a name that is more than 30 characters long).
- Even if parameter name is too long, we still want to leave the GradeBook object in a consistent state. That is, a state in which the object's data member courseName contains a valid value(i.e.,a string of 30 characters or less).
- Thus, we truncate the specified course name and assign the first 30 characters of name to the courseName data member.
- Standard class string provides member function substr (shortfor "substring") that returns a new string object created by copying part of an existing string object.The call in line 28 passes two integers (0 and 25) to name's member function substr.
- The first argument specifies the starting position in the original string. The second argument specifies the number of characters to copy. Therefore,the call in line 28 returns a 30 character substring of name starting at position 0.
- If the member function has to truncate the course name to make it valid, lines 29-30 display a warning message.

Additional Notes on Set Functions

- A public set function such as setCourseName should carefully scrutinize any attempt to modify the value of a data member (e.g., courseName) to ensure that the new value is appropriate for that data item.
- For example,an attempt to set the day of the month to 37 should be rejected, an attempt to set a person's weight to zero or a negative value should be rejected, an attempt to set a grade on an exam to 185 (when the proper range is zero to 100) should be rejected,and so on.
- A class's set functions can return values to the class's clients indicating that attempts were made to assign invalid data to objects of the class.
- A client can test the return value of a set function to determine whether the attempt to modify the object was successful and to take appropriate action.